ShopMate Code Flow Guide

E-commerce Customer Support Assistant

1. High-level architecture

The app is a FastAPI web service with a browser chat UI. The frontend posts questions to /api/chat. The backend classifies the user intent, retrieves relevant policy text, calls mock commerce tools, and optionally uses a Groq LLM to turn retrieved context into a natural-language answer.

2. End-to-end request flow

Browser UI -> app/static/app.js -> POST /api/chat -> app/main.py -> SupportAgent.answer() -> classify intent -> retrieve policy matches -> handle order/return/refund/escalation -> return JSON response.

3. Main code files

app/main.py: creates the FastAPI app, mounts the static frontend, exposes endpoints like /api/chat and /api/health, and serves the UI. app/agent.py: orchestrates the workflow and chooses between direct tool actions and LLM-generated answers. app/knowledge.py: performs retrieval using sentence-transformer embeddings with a TF-IDF fallback. app/tools.py: reads and updates the synthetic order and return records in JSON files. app/config.py: loads environment variables from .env, including the Groq model and key settings. app/static/app.js: sends requests and renders the conversation in the browser.

4. Intent routing

The assistant decides whether a query is about order status, returns, refunds, escalation, or general FAQs. Order and return requests use mocked business logic from the local JSON records. FAQ questions use retrieval-augmented generation to pull relevant help-center text before answering.

5. Data and safety

Data lives under the data folder. The app uses synthetic orders and return requests, not a real database or payment provider. This keeps the project safe for demos while still exercising realistic workflows and logic.

6. Why fallback answers appear

If the Groq call fails, the app catches the exception, logs the error type, and returns the retrieved local knowledge instead. That is why the interface may show: "Groq could not generate a response..." The fallback ensures the service stays usable even without the model being available.

7. Summary

This project is a compact LLM + RAG + rules-based support assistant for ecommerce chat support. The design is easy to follow: input validation in FastAPI, classification and orchestration in the agent,

retrieval in the knowledge layer, mock commerce logic in the tools module, and plain browser-side chat UI in the frontend.